

从 Java 到 JavaScript: 你必须知道的语法差异

糖拌西瓜皮 2026-06-22  19  阅读5分钟

[关注](#)

系列: Java 开发者的 Node.js + TypeScript 之路 (第 2 篇) **适用人群:** 有 Java 基础, 想转向 Node.js 后端开发的开发者

1. 变量声明

let VS const VS var

▼ javascript

[复制代码](#)

```
1 // var - 函数作用域, 有变量提升 (hoisting), 现代代码中不推荐使用
2 var name = 'Java';
3
4 // let - 块级作用域, 类似 Java 的局部变量
5 let count = 0;
6 count = 1; // 可以重新赋值
7
8 // const - 块级作用域, 声明后不能重新赋值 (但对象内容可以修改)
9 const PI = 3.14;
10 const user = { name: 'Tom' };
11 user.name = 'Jerry'; // 对象属性可以修改
12 // user = {}; // 不能重新赋值
```

与 Java 对比

JS	Java	说明
<code>const x = 1</code>	<code>final int x = 1</code>	不可重新赋值
<code>let x = 1</code>	<code>int x = 1</code>	可重新赋值
<code>var x = 1</code>	无直接对应	函数作用域, 避免使用

最佳实践: 默认用 `const` , 需要修改时才用 `let` , 永远不用 `var` 。

2. 数据类型

原始类型 (7 种)

▼ javascript



复制代码

```
1 // 1. number - 不区分整数和浮点数 (与 Java 不同! )
2 const age = 25;
3 const price = 9.99;
4 const inf = Infinity;
5 const notNum = NaN; // Not a Number, 但 typeof 是 'number'
6
7 // 2. string - 单引号、双引号、反引号均可
8 const s1 = 'hello';
9 const s2 = "world";
10 const s3 = `hello ${name}`; // 模板字符串, 可嵌入表达式
11
12 // 3. boolean
13 const isActive = true;
14
15 // 4. undefined - 变量已声明但未赋值
16 let x;
17 console.log(x); // undefined
18
19 // 5. null - 表示"有意为空"
20 const empty = null;
21
22 // 6. symbol - 唯一标识符 (ES6+)
23 const id = Symbol('id');
24
```

```
25 // 7. bigint - 大整数
26 const big = 9007199254740991n;
```

引用类型

▼ javascript



复制代码

```
1 // 对象 (类似 Java 的 Map + 对象)
2 const user = {
3   name: 'Tom',
4   age: 25,
5   greet() {
6     return `Hi, I'm ${this.name}`;
7   }
8 };
9
10 // 数组 (类似 Java 的 List)
11 const nums = [1, 2, 3];
12 nums.push(4); // [1, 2, 3, 4]
13 nums.map(n => n * 2); // [2, 4, 6, 8]
14
15 // 函数也是一等公民 (可以赋值给变量、作为参数传递)
16 const add = (a, b) => a + b;
```

typeof 陷阱

▼ javascript



复制代码

```
1 typeof null // 'object' ← 历史遗留 bug!
2 typeof [] // 'object' ← 数组也是对象
3 typeof NaN // 'number' ← 反直觉
4 Array.isArray([]) // true ← 判断数组用这个
```

3. 类型转换 (与 Java 差异极大)

JavaScript 是弱类型, 会自动进行类型转换 (coercion) 。

▼ javascript



复制代码

```
1 // 隐式转换 (坑多, 要小心)
2 '5' + 3 // '53' (字符串拼接优先)
```

```
3 '5' - 3      // 2      (减法会转数字)
4 true + 1     // 2
5 false == 0   // true   (== 会类型转换)
6 false === 0  // false  (=== 不转换, 推荐)
7
8 // 显式转换
9 Number('123') // 123
10 String(123)  // '123'
11 Boolean(0)   // false
12 Boolean('')  // false
13 Boolean(null) // false
14 Boolean(undefined) // false
15 // 以上为所有 false 值 (加 NaN)
```

最佳实践: 永远使用 `===` 和 `!==`, 不用 `==` 和 `!=`。

4. 函数

函数声明 vs 函数表达式

▼ javascript



复制代码

```
1 // 函数声明 (有提升)
2 function add(a, b) {
3   return a + b;
4 }
5
6 // 函数表达式 (无提升)
7 const add = function(a, b) {
8   return a + b;
9 };
10
11 // 箭头函数 (推荐, 简洁, 不绑定 this)
12 const add = (a, b) => a + b;
13
14 // 默认参数
15 const greet = (name = 'World') => `Hello, ${name}`;
16
17 // 剩余参数 (类似 Java 的可变参数 ...args)
18 const sum = (...nums) => nums.reduce((a, b) => a + b, 0);
```

箭头函数 vs 普通函数

▼ javascript

复制代码

```
1 // 箭头函数没有自己的 this, 继承外层作用域
2 const obj = {
3   name: 'Tom',
4   // 普通函数: this 指向调用者
5   sayHi() {
6     console.log(this.name); // 'Tom'
7   },
8   // 箭头函数: this 指向定义时的外层 (通常是 global 或 undefined)
9   sayHiArrow: () => {
10     console.log(this.name); // undefined
11   }
12 };
```

与 Java 方法对比

特性	Java	JavaScript
函数独立存在	必须在类中	一等公民
默认参数	不支持	支持
函数重载	支持	不支持 (用默认参数/剩余参数替代)
Lambda	(a,b) -> a+b	(a,b) => a+b
this	指向当前实例	取决于调用方式

5. 对象与原型

对象操作

▼ javascript

复制代码

```
1 const user = { name: 'Tom', age: 25 };
2
```

```
3 // 访问属性
4 user.name           // 'Tom'
5 user['name']         // 'Tom' (可用变量作为 key)
6
7 // 可选链 (类似 Java 的 Optional)
8 user?.address?.city // undefined (不会报错)
9
10 // 展开运算符
11 const copy = { ...user, city: 'Beijing' }; // 浅拷贝并扩展
12
13 // 解构赋值
14 const { name, age } = user;
```

原型链 (了解即可)

▼ javascript



复制代码

```
1 // JavaScript 没有传统类继承, 用原型链实现
2 // ES6 的 class 只是语法糖, 底层仍是原型
3
4 class Animal {
5   constructor(name) {
6     this.name = name;
7   }
8   speak() {
9     return `${this.name} makes a sound`;
10  }
11 }
12
13 class Dog extends Animal {
14   speak() {
15     return `${this.name} barks`;
16   }
17 }
18
19 const dog = new Dog('Rex');
20 dog.speak(); // 'Rex barks'
```

Java 开发者注意: JS 的 `class` 是语法糖, 本质是构造函数 + 原型链。在 TypeScript 中会更接近 Java 的类, 但运行时仍是原型。

6. 数组常用方法

▼ javascript



复制代码

```
1  const arr = [1, 2, 3, 4, 5];
2
3  // 遍历 (类似 Java Stream)
4  arr.forEach(x => console.log(x));
5
6  // map (类似 Stream.map)
7  arr.map(x => x * 2);           // [2, 4, 6, 8, 10]
8
9  // filter (类似 Stream.filter)
10 arr.filter(x => x > 3);        // [4, 5]
11
12 // reduce (类似 Stream.reduce)
13 arr.reduce((sum, x) => sum + x, 0); // 15
14
15 // find (类似 Stream.findFirst)
16 arr.find(x => x > 3);          // 4
17
18 // some / every (类似 anyMatch / allMatch)
19 arr.some(x => x > 4);          // true
20 arr.every(x => x > 0);          // true
21
22 // 解构
23 const [first, second, ...rest] = arr;
24 // first=1, second=2, rest=[3,4,5]
```

Java Stream vs JS Array 方法对照

Java Stream**JS Array**`.map()``.map()``.filter()``.filter()``.reduce()``.reduce()``.findFirst()``.find()``.anyMatch()``.some()``.allMatch()``.every()``.collect(Collectors.toList())``.map()` 直接返回数组

7. JSON

▼ javascript



复制代码

```
1 // JSON 是 JS 的子集, 也是前后端数据交换的标准格式
2
3 // 对象 → JSON 字符串 (类似 Jackson 的 writeValueAsString)
4 const json = JSON.stringify({ name: 'Tom', age: 25 });
5 // '{"name":"Tom","age":25}'
6
7 // JSON 字符串 → 对象 (类似 Jackson 的 readValue)
8 const obj = JSON.parse('{"name":"Tom","age":25}');
9 console.log(obj.name); // 'Tom'
```

8. 错误处理

▼ javascript



复制代码

```
1 // 与 Java 的 try-catch 基本一致
2 try {
3   throw new Error('Something went wrong');
```



```
4 } catch (error) {
5   console.error(error.message);
6 } finally {
7   console.log('Always runs');
8 }
9
10 // 自定义错误
11 class NotFoundError extends Error {
12   constructor(resource) {
13     super(`${resource} not found`);
14     this.name = 'NotFoundError';
15   }
16 }
17
18 throw new NotFoundError('User');
```

注意：JS 的 catch 捕获的 error 类型不确定，可能是任何值（不只是 Error 实例）。

9. 作用域与闭包（重要）

作用域链

▼ javascript



复制代码

```
1 // JavaScript 有三种作用域：全局、函数、块级
2
3 let global = '我是全局变量'; // 全局作用域
4
5 function outer() {
6   let outerVar = '我是外层变量'; // 函数作用域
7
8   function inner() {
9     let innerVar = '我是内层变量'; // 块级作用域
10    console.log(global); // 能访问全局
11    console.log(outerVar); // 能访问外层（作用域链向上查找）
12    console.log(innerVar); // 能访问自己
13  }
14
15  inner();
16 }
```

Java 对比: Java 也有类似的作用域规则 (类 > 方法 > 代码块), 但 JS 的函数可以嵌套, 而且内层函数可以「记住」外层变量——这就是闭包。

闭包 (Closure)

一句话理解: 闭包 = 函数 + 它能访问的外部变量。

▼ javascript



复制代码

```
1 // 最简单的闭包
2 function createCounter() {
3   let count = 0; // 这个变量被「关」在函数里
4
5   return {
6     increment() { count++; return count; },
7     decrement() { count--; return count; },
8     getCount() { return count; }
9   };
10 }
11
12 const counter = createCounter();
13 counter.increment(); // 1
14 counter.increment(); // 2
15 counter.getCount(); // 2
16 // count 变量对外不可见, 只能通过方法访问
```

闭包的的实际用途

▼ javascript



复制代码

```
1 // 用途一: 数据私有化 (类似 Java 的 private 字段)
2 function createUser(name) {
3   let loginCount = 0; // 私有变量, 外部无法直接访问
4
5   return {
6     getName() { return name; },
7     login() { loginCount++; },
8     getLoginCount() { return loginCount; }
9   };
10 }
11
12 const user = createUser('Tom');
13 user.login();
14 user.getLoginCount();
```

```
15 user.getLoginCount(); // 2
16 // user.loginCount → undefined (私有)
17
18 // 用途二: 函数工厂
19 function createMultiplier(factor) {
20   return (number) => number * factor; // 记住 factor
21 }
22
23 const double = createMultiplier(2);
24 const triple = createMultiplier(3);
25 double(5); // 10
26 triple(5); // 15
27
28 // 用途三: 缓存 / 记忆化
29 function memoize(fn) {
30   const cache = new Map();
31   return function(...args) {
32     const key = JSON.stringify(args);
33     if (cache.has(key)) return cache.get(key);
34     const result = fn(...args);
35     cache.set(key, result);
36     return result;
37   };
38 }
39
40 const expensiveCalc = memoize((n) => {
41   console.log('计算中...');
42   return n * n;
43 });
44
45 expensiveCalc(5); // 计算中... → 25
46 expensiveCalc(5); // 直接返回 25 (命中缓存)
```

闭包常见陷阱

▼ javascript



复制代码

```
1 // 经典问题: 循环中的闭包
2 // 错误写法
3 for (var i = 0; i < 3; i++) {
4   setTimeout(() => console.log(i), 100);
5 }
6 // 输出: 3, 3, 3 (不是 0, 1, 2! 因为 var 是函数作用域, i 共享)
7
8 // 修正: 用 let (块级作用域, 每次循环有新的 i)
9 for (let i = 0; i < 3; i++) {
10   setTimeout(() => console.log(i), 100);
```

```
11 }  
12 // 输出: 0, 1, 2
```

10. this 完全指南 (最容易晕的部分)

Java 开发者须知

在 Java 中, `this` 永远指向当前对象实例, 非常简单。在 JavaScript 中, `this` 的值取决于函数的调用方式, 不是定义位置!

四种调用方式决定 this

▼ javascript



复制代码

```
1 // 1. 作为对象方法调用 → this = 调用者对象  
2 const user = {  
3   name: 'Tom',  
4   greet() {  
5     console.log(`Hi, I'm ${this.name}`);  
6   }  
7 };  
8 user.greet(); // this = user  
9  
10 // 2. 作为普通函数调用 → this = undefined (严格模式) 或 global  
11 function greet() {  
12   console.log(this); // undefined (严格模式)  
13 }  
14 greet();  
15  
16 // 3. 使用 call / apply / bind 手动指定 → this = 指定的对象  
17 function greet() {  
18   console.log(`Hi, I'm ${this.name}`);  
19 }  
20 greet.call({ name: 'Jerry' }); // 'Hi, I'm Jerry'  
21 greet.apply({ name: 'Tom' }); // 'Hi, I'm Tom'  
22  
23 const boundGreet = greet.bind({ name: 'Bob' });  
24 boundGreet(); // 'Hi, I'm Bob'  
25  
26 // 4. 作为构造函数 (new) → this = 新创建的实例  
27 class User {  
28   constructor(name) {
```

```
29     this.name = name;
30   }
31 }
32 const u = new User('Tom'); // this = u
```

箭头函数与 this

▼ javascript



复制代码

```
1 // 箭头函数没有自己的 this, 继承外层作用域的 this
2
3 class Timer {
4   constructor() {
5     this.seconds = 0;
6   }
7
8   // 错误: 普通函数作为回调, this 丢失
9   startBad() {
10    setTimeout(function() {
11      this.seconds++; // this 不是 Timer 实例!
12      console.log(this.seconds); // NaN
13    }, 1000);
14  }
15
16   // 正确: 箭头函数继承外层的 this
17   startGood() {
18     setTimeout(() => {
19       this.seconds++; // this = Timer 实例
20       console.log(this.seconds); // 1
21     }, 1000);
22   }
23
24   // 也正确: 用 bind 绑定
25   startAlsoGood() {
26     setTimeout(function() {
27       this.seconds++;
28       console.log(this.seconds);
29     }.bind(this), 1000);
30   }
31 }
```

速查表

调用方式	this 指向
obj.method()	obj
func()	undefined （严格模式）
new Class()	新创建的实例
func.call(obj)	obj
() => {}	外层作用域的 this

小结

核心概念	Java	JavaScript
变量	强类型，声明时指定类型	弱类型，let / const
函数	必须在类中	一等公民，可独立存在
类	真实的类	语法糖（原型链）
null/undefined	只有 null	null + undefined
相等比较	==	=== （推荐）

下一篇预告：ES6+ 核心特性速查：解构、箭头函数、Promise 与模块化

标签： JavaScript Java Node.js